Wavefront: A TLM-based Simulation Platform for Teaching Basic Wave Propagation Principles

Leo Kling[1,†], Jonathan Kron[1,†], Hendrik Himmelein[1,2,*], Christoph Pörschmann[1]
[1] *Institute of Computer and Communication Technology (ICCT), Technische Hochschule Köln, 50679 Köln*
[2] *Audio Communication Group, Technische Universität Berlin, 10623 Berlin*
[†] *Both authors contributed equally to this work*
[*] *Corresponding Author: hendrik.himmelein@th-koeln.de*

Introduction

Visualizing acoustic wave propagation for educational purposes can help students understand acoustic phenomena such as diffraction, refraction, and reflection, or the superposition of multiple sound sources. These concepts can be difficult to grasp, especially for students who are new to the field of acoustics. Moreover, existing simulation tools often suffer from excessive complexity, poor usability, a lack of real-time functionalities, or often require advanced programming skills, limiting their practicality and adoption for educational purposes [1, 2]. At TH Köln, similar tools have been used for quite some time, and through their use, the challenges mentioned above have been identified, as well as opportunities for improvement to enhance the usability and thus the learning experience. With these prerequisites in mind, we aim to develop a modern learning platform with an emphasis on an improved user interface (UI) and user experience (UX), making the tool more accessible, efficient, and enjoyable to experience basic wave propagation.

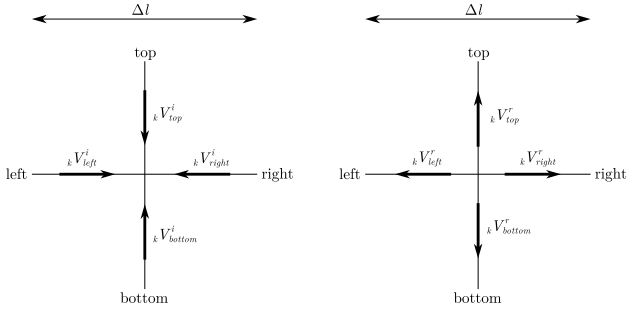
Thus, we introduce *Wavefront*, a novel educational tool that facilitates hands-on teaching and learning of the fundamental concepts of acoustic wave propagation. We chose the TLM (Transmission-Line Matrix) method for simulating wave propagation in 2D space because it is well suited for real-time visualization and is easy and efficient to implement [3]. Moreover, it operates in the time domain, which makes it easier to comprehend compared to frequency-domain approaches.

We have implemented Wavefront using the Rust programming language, which offers high performance and memory safety. The code is open source and released under a permissive license, allowing users to study and customize the source code to suit their needs. Additional features will be implemented in the future, and active user participation is encouraged.

Method

Acoustic sound radiation in space can generally be described by the wave equation, which can be solved analytically for simple cases. For complex scenarios, however, it is often not possible to find closed-form mathematical solutions. Over the last few decades, a number of computational approaches have been developed to find approximate solutions. These include the finite-difference method (FDM), the finite element method (FEM), the boundary element method (BEM) and the transmission-line matrix method (TLM). For Wavefront, we chose the TLM method for its algorithmic simplicity, accuracy, and low computational requirements [3].

The TLM method was originally introduced by Johns and Beurle [4] as an implementation of an electrical network model to solve an electromagnetic field problem. It is based on Huygens' model of wave propagation, which states that each point of an existing wavefront can be considered as a secondary spherical source emitting at the same velocity and frequency as the original wave. To implement a TLM network, both the spatial and time domains are discretized. The spatial discretization produces a grid of TLM nodes (also called cells), each connected by eight transmission lines. Four of these lines carry incident pulses, while the remaining four carry scattered pulses.



(a) Incidence pulses **(b)** Scattering pulses

Figure 1: TLM node from [5]

The distance between two adjacent TLM nodes (the elementary space step) is denoted by Δl. The time step Δt at which the TLM solution is advanced per simulation step is equal to the time taken for a pulse to travel from one TLM node to an adjacent one [6]. Wavefront allows the user to select a value for Δl to simulate spatial domains of different sizes. The time step is then calculated as

$$\Delta t = \frac{\Delta l}{343.2\mathrm{m\,s^{-1}} \cdot \sqrt{2}} \tag{1}$$

to adjust the TLM model to the speed of sound and to compensate for the discretization error [3].

Each iteration of the TLM method consists of two processes:

Scattering: calculate reflection pulses from incident pulses for each node

Connecting: propagate reflection pulses to adjacent nodes

Of both processes, only the connection process consumes simulation time, as the impulse needs time to travel through the transmission lines. The scattering process is assumed to be instantaneous.

For the calculation of the reflection pulses, the equation

$$ {}_k\boldsymbol{V}^r = \boldsymbol{S} \cdot {}_k\boldsymbol{V}^i \tag{2} $$

must be solved. ${}_k\boldsymbol{V}^r$ and ${}_k\boldsymbol{V}^i$ denote the reflection and incident pulses at the time step k defined by

$$ \boldsymbol{V} = \begin{pmatrix} V_{bottom} \\ V_{left} \\ V_{top} \\ V_{right} \end{pmatrix} \tag{3} $$

and $\boldsymbol{S}$ is the scattering matrix

$$ \boldsymbol{S} = \begin{pmatrix} -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & -1 \end{pmatrix}. \tag{4} $$

The connection between incident and reflection pulses is defined as

$$ \begin{aligned} {}_{k+1}V^i_{bottom}(x,y) &= {}_kV^r_{top}(x,y-1) \\ {}_{k+1}V^i_{left}(x,y) &= {}_kV^r_{right}(x-1,y) \\ {}_{k+1}V^i_{top}(x,y) &= {}_kV^r_{bottom}(x,y+1) \\ {}_{k+1}V^i_{right}(x,y) &= {}_kV^r_{left}(x+1,y), \end{aligned} \tag{5} $$

where x and y denote the spatial position of a TLM node.

Implementation

Prior to the current implementation in Rust, we prototyped Wavefront in Java and the Godot engine to evaluate the feasibility of the TLM method for real-time wave propagation. The Java prototype was developed as a proof of concept to explore the inner workings of the TLM algorithm. Godot was chosen because of its 2D game engine and UI capabilities, which allowed for easy visualization. However, both prototypes were limited in performance and scalability, motivating the development of the current Rust implementation.

Finally, we chose Rust as a modern, memory-safe C++ alternative [7]. Rust also relies on manual memory management (as opposed to Java), which is important for the performance of the simulation. The Bevy game engine [8] was used for rendering and the egui library for the user interface [9]. Both support cross-platform development for Windows, macOS, Linux, and also the Web by utilizing WebAssembly.

Bevy makes use of an ECS (Entity Component System), which allows for an efficient and scalable software architecture [10]. Each entity (sources, microphones, or walls) is made up of components that define their behavior. For example, a source entity has a position, a frequency, and an amplitude component. The simulation system then iterates over all entities and updates their state in parallel.

The simulation is executed on a separate thread from the UI to ensure real-time performance and decoupling of simulation and UI frame rate, as egui is an immediate mode GUI library. The simulation is divided into four main steps: calculating reflection pulses, applying source pulses, writing into microphones, and feeding forward pulses. The simulation loop is shown in Figure 4.

To represent the TLM grid in memory, it is split into three one-dimensional arrays, which can be thought of as different "layers". One stores all the current cells, one stores the next cells, and one stores the calculated pressure values. This allows for an efficient way to iterate over the arrays in parallel without any race conditions, with the trade-off being higher memory usage. All arrays are initialized with size

$$ (w_s + 2 \cdot w_b) \cdot (h_s + 2 \cdot h_b), \tag{6} $$

where w_s and w_b are the width and h_s and h_b are the height of the simulation area and the boundary layer, respectively. The boundary layer can be interpreted as an absorber that is used to attenuate the reflection of sound waves at the boundaries of the simulation area and is initialized with a width of 50. The simulation width and height are set to 700 by default.

The absorbing properties of the boundary layer are utilized to approximate a free-field simulation environment. The boundary layer operates by gradually reducing the coefficients $\boldsymbol{S}_{1,1}$, $\boldsymbol{S}_{2,2}$, $\boldsymbol{S}_{3,3}$, $\boldsymbol{S}_{4,4}$ within their respective trapezoidal regions (see Figure 3) to zero as they approach the outer edge of the boundary layer by assigning them a value of

$$ 1 - \left(\frac{d}{w_b}\right)^5, \tag{7} $$

where d is the distance from the TLM node in the boundary layer to the outer edge of the simulation area. Specifically, in the top boundary layer, only $\boldsymbol{S}_{1,1}$ is modified, in the left boundary layer, only $\boldsymbol{S}_{4,4}$ is affected, and so forth.

During the `Calculate Reflection Pulses` step, all cells within both the simulation and boundary regions are updated. Each cell can either belong to the simulation region, the boundary region, or represent a wall. If a cell is identified as a wall edge, then Equation 2 can

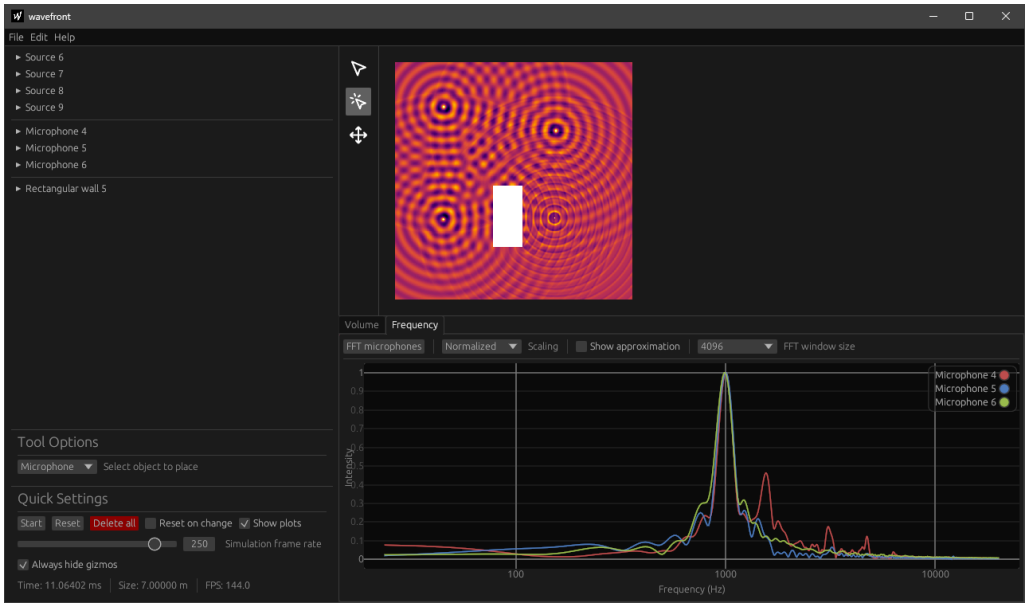


Figure 2: Wavefront interface implemented using the egui library

be simplified to

$$ {}_k\boldsymbol{V}^r = \boldsymbol{R} \cdot {}_k\boldsymbol{V}^i, \tag{8} $$

where $\boldsymbol{R}$ is a four-dimensional vector representing the reflection coefficients of the wall in each direction. In our case, all components of $\boldsymbol{R}$ are equal. If the cell is identified to lie inside a wall, then $\boldsymbol{R}$ is set to zero. This ensures that wave propagation is entirely prevented within non-hollow walls.

During the `Apply Source Pulses` step, the contributions of all active point sources are introduced into the simulation domain. Each point source injects either a sine wave, a Gaussian impulse, or a white noise signal into its respective location.

Features

Wavefront includes several key features designed to enhance its usability as an educational tool, such as real-time visualization of wave propagation, simulation of multiple sound sources, obstacle placement, and cross-platform compatibility. Furthermore, the platform supports execution within a web browser via WebAssembly, removing installation barriers and ensuring accessibility across different operating systems.

To simplify analysis, Wavefront includes virtual microphone placement, allowing users to sample and examine acoustic signals at different spatial positions. This feature is particularly valuable for studying sound field characteristics and verifying theoretical predictions.

In addition, modern convenient features such as screenshot functionality, seamless loading and saving, an undo/redo system, and simulation color themes are implemented for quick and easy use.

The implementation of loading external audio files for use as sound sources has been successfully completed, as has the export functionality of microphone recordings.

Frequency analysis functions for microphone recordings have also been integrated, offering flexibility in terms of different FFT (Fast Fourier Transform) sizes and the option to apply smoothing to the resulting frequency curve. Despite this, certain frequency artifacts have been observed, starting at 5 kHz and becoming more prevalent at higher frequencies. We hypothesize the discretized nature of the TLM algorithm as the root cause, but further investigation is required to identify potential solutions.

Evaluation

Wavefront was evaluated in two acoustic lab sessions at TH Köln. Students found the visual simulations intuitive, enhancing their understanding and engagement. Feedback from students and teaching staff confirmed its

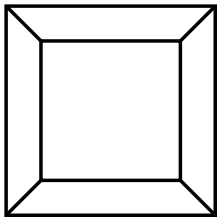


Figure 3: Simulation area and boundary layers

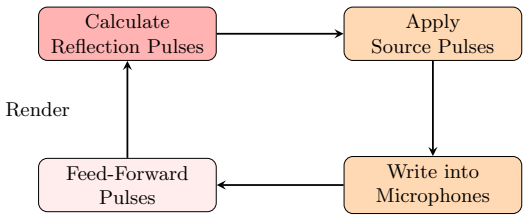


Figure 4: Simulation processing flow

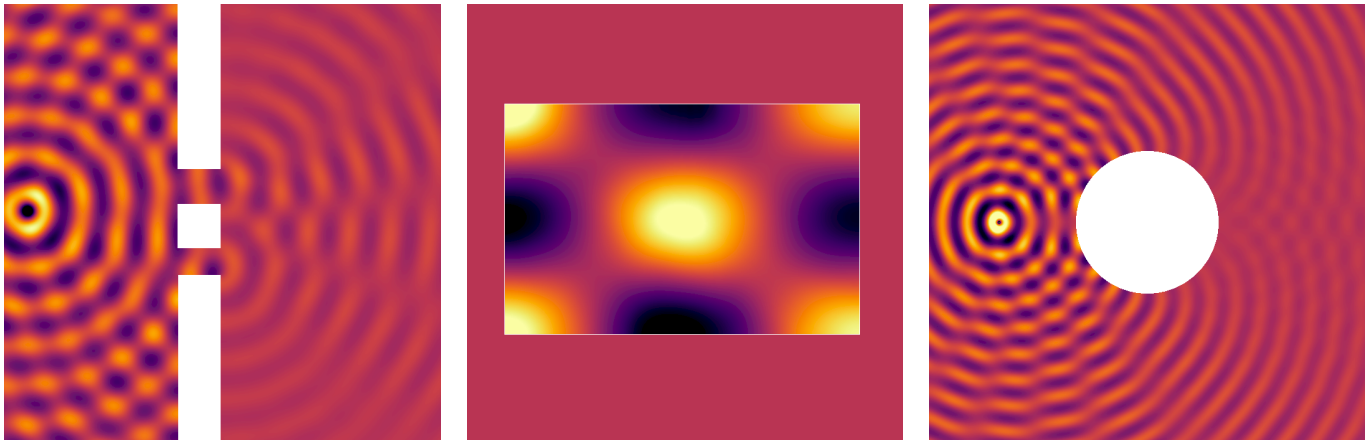


Figure 5: Simulation results: Double-slit interference (left), room modes (middle), acoustic shadow (right)

effectiveness for teaching wave propagation concepts.

During these student trials, several wave propagation phenomena were evaluated, including wave reflection, diffraction, and room modes. The room modes were first determined mathematically, then simulated with Wavefront, and finally verified experimentally through measurements in a scaled physical model to verify their accuracy. A simulation of such a mode is shown in Figure 5.

Wavefront performed well across a variety of CPU architectures, where the simulation consistently achieved frame rates exceeding 400 updates per second. On lower-end laptop hardware, the simulation maintained usable performance, ensuring adequate responsiveness and computational efficiency even on less powerful systems. Additionally, it performed well on the Web using WebAssembly, including mobile devices.

Conclusion

We introduced Wavefront, a lightweight and efficient tool for simulating acoustic sound fields. It is well suited for illustrative simulations for educational purposes and for quickly visualizing acoustic phenomena. Wavefront offers real-time performance, cross-platform compatibility, and ease of use while remaining open source. We hope that this open-source project will find its way into university education and become a valuable tool for teaching and learning. We welcome contributions from the community and look forward to developing Wavefront together. The source code, as well as precompiled binaries for Mac, Windows, and Linux, are available on GitHub: `https://github.com/AudioGroupCologne/wavefront`

References

[1] J. Petrosino, L. F. Landini, G. A. Lizaso, A. I. Kuri, and I. Canalis, "MATLAB-based simulation software as teaching aid for physical acoustics," *Proceedings of the ICA 2019 and EAA Euroregio : 23rd International Congress on Acoustics*, vol. integrating 4th EAA Euroregio 2019 : 9-13 September 2019, 2019. DOI: 10.18154/RWTH-CONV-239863.

[2] A. Rocha, S. Mota, and C. Ribeiro, "Software Tools for Teaching Wave Propagation in Transmission Lines [Education Corner]," *IEEE Antennas and Propagation Magazine*, vol. 59, no. 3, pp. 118–127, Jun. 2017. DOI: 10.1109/MAP.2017.2686083.

[3] A. Vázquez Giner, "Design and Implementation of a C++/Qt Based Simulation Software for Sound Wave Propagation," M.S. thesis, Technische Hochschule Köln, Jun. 2016.

[4] P. Johns and R. Beurle, "Numerical solution of 2-dimensional scattering problems using a transmission-line matrix," *Proceedings of the Institution of Electrical Engineers*, vol. 118, no. 9, pp. 1203–1208, 1971. DOI: 10.1049/piee.1971.0217.

[5] Y. Kagawa, T. Tsuchiya, T. Hara, and T. Tsuji, "Discrete huygens' modelling simulation of sound wave propagation in velocity varying environments," *Journal of Sound and Vibration*, vol. 246, pp. 419–439, Sep. 2001. DOI: 10.1006/jsvi.2001.3637.

[6] A. Giannopoulos, "The investigation of transmission-line matrix and finite-difference time-domain methods for the forward problem of ground probing radar.," en, Ph.D. dissertation, University of York, Mar. 1997. [Online]. Available: https://etheses.whiterose.ac.uk/id/eprint/2443/.

[7] S. Klabnik and C. Nichols, *The Rust Programming Language, 2nd Edition*, en. No Starch Press, Feb. 2023, Google-Books-ID: SE2GEAAAQBAJ, ISBN: 978-1-71850-310-6.

[8] "Bevy Engine." en. (Mar. 21, 2025), [Online]. Available: https://bevyengine.org/ (visited on 03/23/2025).

[9] E. Ernerfeldt, *Emilk/egui*, Mar. 23, 2025. [Online]. Available: https://github.com/emilk/egui (visited on 03/23/2025).

[10] S. Bilas, "A data-driven game object system," in *Game Developers Conference Proceedings*, vol. 2, 2002.